

Compilation and Program Analysis (#10) : Functions: semantics

Ludovic Henrio

Master 1, ENS de Lyon et Dpt Info, Lyon1

2025-2026



Approach and objective of the course

- Give a semantics to functions in an imperative setting.
inspired by what can be found in the literature
- Explore the different ways to deal with variables, different environments of execution, ...
- Big step and small step operational semantics
- Observe that things are more complex but also more constructive than with the λ -calculus.

Note on organisation:

1: Course + **live proofs** ;

2: **exercises and proofs done after the course**

1 Operational Semantics for functions

- Big-step semantics: first solution
- Big-step semantics: second solution
- Big-step semantics: third solution
- Small-step semantics based on third solution

2 Safety of the type-system

Mini-While Syntax 1/2

Expressions:

$$e ::= c \mid e + e \mid e \times e \mid \dots$$

Mini-while:

$S(Smt) ::=$	$x := expr$	assign
	$ \quad x := f(e_1, \dots, e_n)$	simple function call
	$ \quad skip$	do nothing
	$ \quad S_1; S_2$	sequence
	$ \quad \text{if } b \text{ then } S_1 \text{ else } S_2$	test
	$ \quad \text{while } b \text{ do } S \text{ done}$	loop

Mini-While Syntax 2/2

[NEW] Programs with function definitions and global variables

$Prog$	$::= D \text{ FunDef } Body$	Program
$Body$	$::= D; S$	Function/main body
D	$::= var\ x : \tau \mid D; D$	Variable declaration
$FunDef$	$::= \tau\ f(x_1 : \tau_1, \dots, x_n : \tau_n)\ Body; return\ e$ $\mid FunDef\ FunDef$	Function def

Note/discussion: to simplify syntax and semantics:

- 1) function call is not an expression but a special statement.
- 2) return only appears at the end of the function definition (enforced by syntax).

Dealing with variable declaration and store management

Variable declaration: $Vars(D)$ is the set of variables declared by D .

Reminder: we could use it to initialize the local memory when needed and ensure progress (see typing course).

We define a global store update:

$$\sigma'[X \mapsto \sigma](x) = \begin{cases} \sigma(x) & \text{if } x \in X \\ \sigma'(x) & \text{else} \end{cases}$$

This will be used to restore part of the store to a previous value.

Function table

For each function declared with name f we have $params(f)$ the list of parameter variables, $ret(f)$ the expression in the return statement, and $body(f)$ the function body.

This could be formally defined as a function table Φ that is given as parameter of the semantics, i.e.: change the signature into $\Phi \vdash (S, \sigma) \longrightarrow \sigma'$ and use Φ to obtain $ret(f)$, $params(f)$, and $body(f)$.

Here we suppose that the functions ret , $params$ and $body$ are globally known.

Big step semantics (old) 1/2

$$\longrightarrow: Stm \rightarrow (State \rightarrow State)$$

$$(x := e, \sigma) \longrightarrow \sigma[x \mapsto Val(e, \sigma)]$$

$$(skip, \sigma) \longrightarrow \sigma$$

$$\frac{(S_1, \sigma) \longrightarrow \sigma' \quad (S_2, \sigma') \longrightarrow \sigma''}{((S_1; S_2), \sigma) \longrightarrow \sigma''}$$

Big step semantics (old) 2/2

$$\frac{Val(b, \sigma) = tt \quad (S_1, \sigma) \longrightarrow \sigma'}{(if\ b\ then\ S_1\ else\ S_2, \sigma) \longrightarrow \sigma'}$$

$$\frac{Val(b, \sigma) = ff \quad (S_2, \sigma) \longrightarrow \sigma'}{(if\ b\ then\ S_1\ else\ S_2, \sigma) \longrightarrow \sigma'}$$

$$\frac{Val(b, \sigma) = tt \quad (S, \sigma) \longrightarrow \sigma' \quad (while\ b\ do\ S\ done, \sigma') \longrightarrow \sigma''}{(while\ b\ do\ S\ done, \sigma) \longrightarrow \sigma''}$$

$$\frac{Val(b, \sigma) = ff}{(while\ b\ do\ S\ done, \sigma) \longrightarrow \sigma}$$

Evaluation of program: Let $D \text{ FunDef } D'; S$ be a program, its evaluation is $(S, \emptyset) \longrightarrow \sigma'$

Additional simplification

We only consider to functions that have a simple parameter for the semantics.

We use $f(e)$ instead of $f(e_1, \dots, e_n)$. Thus $params(f)$ is an array of a single element $[x]$

Extending to multiple parameters raises no particular issue but entails more manipulation of indices.

In the examples we may use several parameters.

1 Operational Semantics for functions

- Big-step semantics: first solution
- Big-step semantics: second solution
- Big-step semantics: third solution
- Small-step semantics based on third solution

Big step semantics (NEW) – First solution

Heavy manipulation of stores:

$$\frac{\begin{array}{l} \text{body}(f) = D_f; S_f \\ \text{bind}_1(f, e, \sigma) = (S, \sigma') \quad (S, \sigma') \longrightarrow \sigma'' \quad v = \text{Val}(\text{ret}(f), \sigma'') \end{array}}{(x := f(e), \sigma) \longrightarrow \sigma''[(\text{Vars}(D_f) \cup \text{params}(f)) \mapsto \sigma, x \mapsto v]}$$

Where:

$$\text{bind}_1(f, e, \sigma) = (S_f, \sigma[x' \mapsto v'])$$

with $\text{body}(f) = D_f; S_f$ $\text{params}(f) = [x']$ $\text{Val}(e, \sigma) = v'$

Initial configuration:

$$(S_m, \emptyset) \text{ for program } D \quad \text{FunDef } D'; S$$

An example

Evaluate the following program:

```
int x
int f(int x) {
  int y;
  x:=1;
  y:=2;
  return x+1;
}
{
  int y;
  x:=0;
  y:=0;
  y:=f(3);
}
```

Evaluation of this first solution

Notes:

- call-by-value,
- parameters in store,
- Non-trivial store restoration.

Variables: What happens with variables that have the same name? local vs. local? global vs. local? recursive invocations? **discussion live**

Problem: $\sigma'[(Vars(D_f) \cup params(f)) \mapsto \sigma]$ is used to restore the store as it was before the invocation.

This is difficult to reason about, and far from any implementation.

But not much changes to the other rules, the configurations are unchanged.

It is not easy to adapt to small-step. **why?**

1 Operational Semantics for functions

- Big-step semantics: first solution
- **Big-step semantics: second solution**
- Big-step semantics: third solution
- Small-step semantics based on third solution

Big step semantics (NEW) – Second solution

Renaming and fresh variables:

$$\frac{\text{bind}_2(f, e', \sigma) = (S', \sigma', e) \quad (S', \sigma') \longrightarrow \sigma'' \quad v = \text{Val}(e, \sigma'')}{(x := f(e'), \sigma) \longrightarrow \sigma''[x \mapsto v]}$$

$$\text{bind}_2(f, e', \sigma) = (S'_f, \sigma[z \mapsto v'], e)$$

where

$$\text{body}(f) = D_f; S_f \quad \text{params}(f) = [x'] \quad \text{Vars}(D_f) = \{y_1..y_k\} \quad z \text{ fresh}$$

$$\forall i \in [1..k]. t_i \text{ fresh} \quad \text{Val}(e', \sigma) = v' \quad S'_f = S_f[z/x'][t_1/y_1]..[t_k/y_k]$$

$$e = \text{ret}(f)[z/x'][t_1/y_1]..[t_k/y_k]$$

and fresh means not in σ (and not among the other fresh variables)

An example

Evaluate the following program:

```
int x
int f(int x) {
  int y;
  x:=1;
  y:=2;
  return x+1;
}
{
  int y;
  x:=0;
  y:=0;
  y:=f(3);
}
```

Evaluation of this Second solution

Note: We can see in the store the values of variables that are hidden by the current context but we cannot access them because of renaming.

There are also entries in the stores for function frames that are finished but are still visible, these variables cannot be accessed any more.

Variables: The store grows (unbounded) but we could easily remove useless variables, (detecting useless variables is possible but not trivial).

Problem: variable substitution difficult to reason about, e.g. for mechanical proofs (more difficult if recursive blocks). But no complex store manipulation.

Adaptation to small-step semantics is easy

1 Operational Semantics for functions

- Big-step semantics: first solution
- Big-step semantics: second solution
- **Big-step semantics: third solution**
- Small-step semantics based on third solution

Big step semantics (NEW) – Third solution (1/2)

A store and a stack: $\longrightarrow: Stm, Stack, Store \rightarrow Stack, Store$

Where $Stack : Var \rightarrow address$ and $Store : address \rightarrow Val$

$$\frac{\begin{array}{l} bind_3(f, e', \Sigma, sto) = (S', \Sigma', sto') \\ (S', \Sigma', sto') \longrightarrow (\Sigma'', sto'') \quad v = Val(ret(f), sto'' \circ \Sigma'') \end{array}}{(x := f(e'), \Sigma, sto) \longrightarrow (\Sigma, sto''[\Sigma(x) \mapsto v])}$$

$bind_3(f, e', \Sigma, sto) = (S_f, \Sigma', sto[\ell \mapsto v'])$ where

$body(f) = D_f; S_f \quad params(f) = [x'] \quad Vars(D_f) = \{y_1..y_k\} \quad \ell \text{ fresh}$

$\ell'_1..\ell'_k \text{ fresh} \quad Val(e', sto \circ \Sigma) = v' \quad \Sigma' = \Sigma[x' \mapsto \ell][y_1 \mapsto \ell'_1]$

where fresh means location not in sto (and not among the other fresh locations picked)

!! Access to store must be changed everywhere. See next slide!!

Big step semantics (NEW) – Third solution (2/2)

We now have to deal with two levels for memory addressing, this modifies the whole semantics: $Val(e_i, \sigma)$ replaced by $Val(e_i, sto \circ \Sigma)$ everywhere.

And we have a new assign rule: $(x := e, \Sigma, sto) \longrightarrow \Sigma, sto[\Sigma(x) \mapsto Val(e, sto \circ \Sigma)]$

- the stack is “unstacked” after method invocation.
- in $\Sigma' = \Sigma[x \mapsto \ell][y_1 \dots]$, Σ is too big: only global variables are needed.
- Works with recursively defined blocks with local variables (**how?**)
- Memory grows unbounded but Σ is bounded; we could remove useless locations (gc or manually), they are easy to spot.
- Have to deal with two levels for memory addressing but this is closer to reality (e.g. statements are not modified upon function call).
- **What is a good initial configuration (in particular Σ)?**

An example

Evaluate the following program:

```
int x
int f(int x) {
  int y;
  x:=1;
  y:=2;
  return x+1;
}
{
  int y;
  x:=0;
  y:=0;
  y:=f(3);
}
```

1 Operational Semantics for functions

- Big-step semantics: first solution
- Big-step semantics: second solution
- Big-step semantics: third solution
- Small-step semantics based on third solution

Structural Op. Semantics (SOS = small step) for mini-while (OLD)

$$(x := a, \sigma) \Rightarrow \sigma[x \mapsto Val(a, \sigma)]$$

$$(\text{skip}, \sigma) \Rightarrow \sigma$$

$$\frac{(S_1, \sigma) \Rightarrow \sigma'}{((S_1; S_2), \sigma) \Rightarrow (S_2, \sigma')} \quad \frac{(S_1, \sigma) \Rightarrow (S'_1, \sigma')}{((S_1; S_2), \sigma) \Rightarrow (S'_1; S_2, \sigma')}$$

$$\frac{Val(b, \sigma) = tt}{(\text{if } b \text{ then } S_1 \text{ else } S_2, \sigma) \Rightarrow (S_1, \sigma)}$$

$$\frac{Val(b, \sigma) = ff}{(\text{if } b \text{ then } S_1 \text{ else } S_2, \sigma) \Rightarrow (S_2, \sigma)}$$

the challenge with small step semantics

Writing a SOS is often desirable but with our functions it raises several challenges. Indeed call and return cannot be done in the same rule, and thus:

- Previous state of the memory (before function call is not accessible in the rule for returning from the function),
- The point where the caller was before the call is also lost.

General solution: add thee information and define an extended syntax for “configurations” i.e. state of the program execution at runtime.

OLD SOS with new store structure – Principle

New configuration: $(Stm, ??, Stack, Store)$. $??$ defined later.

$$(x := e, ??, \Sigma, \mathbf{sto}) \Rightarrow (\Sigma, \mathbf{sto}[\Sigma(x) \mapsto Val(e, \mathbf{sto} \circ \Sigma)])$$

$$(\text{skip}, ??, \Sigma, \mathbf{sto}) \Rightarrow (\Sigma, \mathbf{sto})$$

$$(S_1, ??, \Sigma, \mathbf{sto}) \Rightarrow (\Sigma', \mathbf{sto}')$$

$$\frac{(S_1, ??, \Sigma, \mathbf{sto}) \Rightarrow (\Sigma', \mathbf{sto}')}{((S_1; S_2), ??, \Sigma, \mathbf{sto}) \Rightarrow (S_2, ??, \Sigma', \mathbf{sto}')}$$

$$(S_1, ??, \Sigma, \mathbf{sto}) \Rightarrow (S'_1, ??, \Sigma', \mathbf{sto}')$$

$$\frac{(S_1, ??, \Sigma, \mathbf{sto}) \Rightarrow (S'_1, ??, \Sigma', \mathbf{sto}')}{((S_1; S_2), ??, \Sigma, \mathbf{sto}) \Rightarrow ((S'_1; S_2), ??, \Sigma', \mathbf{sto}')}$$

$$Val(b, \mathbf{sto} \circ \Sigma) = tt$$

$$\frac{Val(b, \mathbf{sto} \circ \Sigma) = tt}{(\text{if } b \text{ then } S_1 \text{ else } S_2, ??, \Sigma, \mathbf{sto}) \Rightarrow (S_1, ??, \Sigma, \mathbf{sto})}$$

$$Val(b, \mathbf{sto} \circ \Sigma) = ff$$

$$\frac{Val(b, \mathbf{sto} \circ \Sigma) = ff}{(\text{if } b \text{ then } S_1 \text{ else } S_2, ??, \Sigma, \mathbf{sto}) \Rightarrow (S_2, ??, \Sigma, \mathbf{sto})}$$

Small step semantics, based on third solution

?? is used to remember the execution contexts: it is a list of $(Stack, Stm)$. Let $::$ be the list constructor. Ctx is a list of $(Stack, Stm)$. In the big step semantics this is not needed because the inference is more complex (and remembers contexts)

CALL

$$\frac{bind_3(f, e', \Sigma, sto) = (S', \Sigma', sto')}{(x := f(e'); S, Ctx, \Sigma, sto) \Rightarrow (S', (\Sigma, x := R(f); S) :: Ctx, \Sigma', sto')}$$

$x := f(e'); S$ is (by definition) the whole current statement (body). $R(f)$ is just a marker that remembers the name of the function called (and the calling point).

$bind_3$ is already defined.

SOS with new store structure and contexts

$$\begin{array}{c}
 (x := e, Ctx, \Sigma, \textcolor{green}{sto}) \Rightarrow (Ctx, \Sigma, \textcolor{green}{sto}[\Sigma(x) \mapsto \textcolor{green}{Val}(e, \textcolor{green}{sto} \circ \Sigma)]) \\
 \frac{(S_1, Ctx, \Sigma, \textcolor{green}{sto}) \Rightarrow (Ctx, \Sigma', \textcolor{green}{sto}')}{((S_1; S_2), Ctx, \Sigma, \textcolor{green}{sto}) \Rightarrow (S_2, Ctx, \Sigma', \textcolor{green}{sto}')} \\
 \frac{(S_1, Ctx, \Sigma, \textcolor{green}{sto}) \Rightarrow (S'_1, Ctx, \Sigma', \textcolor{green}{sto}')}{((S_1; S_2), Ctx, \Sigma, \textcolor{green}{sto}) \Rightarrow (S'_1; S_2, Ctx, \Sigma', \textcolor{green}{sto}')}
 \end{array}$$

And a new **rule** for return (when current computation finished)

$$\frac{v = \textcolor{green}{Val}(\textcolor{green}{ret}(f), \textcolor{green}{sto} \circ \Sigma')}{((\Sigma, x := R(f); S) :: Ctx, \Sigma', \textcolor{green}{sto}) \Rightarrow (S, Ctx, \Sigma, \textcolor{green}{sto}[\Sigma(x) \mapsto v])}$$

if, skip, and while rules are trivially adapted

Initial configuration like in the third solution for big step, with an empty Ctx .

What do we have at the end of the execution?

An example

Evaluate the following program:

```
int x
int f(int x) {
  int y;
  x:=1;
  y:=2;
  return x+1;
}
{
  int y;
  x:=0;
  y:=0;
  y:=f(3);
}
```

- 1 Operational Semantics for functions
- 2 Safety of the type-system

[REMINDER] Safety = well typed programs do not go wrong

In case of a small-step semantics safety relies on two lemmas:

Well-type programs run without error

Lemma (progression for mini-while)

*If $\Gamma \vdash (S, \sigma)$, then there exists S', σ' such that $(S, \sigma) \Rightarrow (S', \sigma')$
OR there exists σ' such that $(S, \sigma) \Rightarrow \sigma'$.*

... and remain well-typed

Lemma (preservation)

If $\Gamma \vdash (S, \sigma)$ and $(S, \sigma) \Rightarrow (S', \sigma')$ then $\Gamma \vdash (S', \sigma')$.

Note: (S, σ) cannot be a final configuration. Γ never changes (defined by declarations)

Recall the property for expression evaluation: if σ and Γ agree on all variables the valuation of the expression is of the right type.

[Reminder] Typing rules (simplified with 1 parameter)

Typing of statements has the form : $\Gamma, \Gamma_f \vdash S$ Where Γ : map that defines the variable types, Γ_f : function map, S statement.

$$\begin{array}{c}
 \Gamma_D(D) + \Gamma_D(D_m), \Gamma_f(Fundef) \vdash S \\
 \forall (\tau \ f(x_1 : \tau_1) \ D_f; S_f; \text{return } e) \in Fundef. \\
 \hline
 \Gamma_D(D) + \Gamma_l \vdash e : \tau \wedge \Gamma_D(D) + \Gamma_l, \Gamma_f(Fundef) \vdash S_f \quad \text{with } \Gamma_l = \Gamma_D(x_1 : \tau_1; D_f) \\
 \hline
 \vdash D \ Fundef \ D_m; S
 \end{array}$$

$\Gamma_g + \Gamma_l$ overrides Γ_g with Γ_l , i.e. $(\Gamma_g + \Gamma_l)(x)$ is $\Gamma_l(x)$ if it is defined and $\Gamma_g(x)$ else.

CALL

$$\begin{array}{c}
 \Gamma_f(f) = \tau_1 \rightarrow \tau \quad \Gamma \vdash e : \tau_1 \quad \Gamma \vdash x : \tau \\
 \hline
 \Gamma, \Gamma_f \vdash x := f(e)
 \end{array}$$

State type safety and prove it for functions

Slight change in the correctness wrt store, it cannot be:

$$\Gamma \vdash (S, \sigma) \iff (\Gamma \vdash S \wedge \forall x. \emptyset \vdash \sigma(x) : \tau \iff \Gamma(x) = \tau)$$

any more!

Attempt 1:

$$\begin{aligned} \Gamma, \Gamma_f \vdash (S, Ctx, \Sigma, \sigma) &\iff (\Gamma, \Gamma_f \vdash S \wedge \forall x. (\emptyset \vdash \sigma(\Sigma(x)) : \tau \iff \Gamma(x) = \tau) \\ &\wedge \forall (\Sigma', y := R(f); S') \in Ctx. \mathbf{\Gamma}, \Gamma_f \vdash S' \wedge \forall x. (\emptyset \vdash \sigma(\Sigma'(x)) : \tau \iff \mathbf{\Gamma}(x) = \tau) \\ &\wedge \exists \tau'. \Gamma_f(f) = \tau_1(..\tau_n) \rightarrow \tau' \wedge \mathbf{\Gamma}(y) = \tau' \\ &\wedge \text{all function bodies are well-typed (cf rule)}) \end{aligned}$$

What is wrong?

Preservation and progress

- . **On board: adaptation of the theorem + proof “sketch” for one or 2 cases**

Recall we have to deal with variable initialisation.

First, we need also a typing rule for $x := R(f)$!

[Bonus 1] Dealing with non-terminal return

In big step semantics: One simple solution is to interrupt big step and get back to function call

$$\frac{v = Val(e, \Sigma)}{(return\ e, \Sigma) \longrightarrow (return\ v, \Sigma)}$$

Need to deal with sequence:

$$\frac{(S_1, \Sigma) \longrightarrow (return\ v, \Sigma')}{((S_1; S_2), \Sigma) \longrightarrow (return\ v, \Sigma')}$$

In small step, we can trigger the new rule for return at this point.

[Bonus 2] One semantics from the literature: imperative objects

From: Gordon A.D., Hankin P.D., Lassen S.B. (1997) Compilation and equivalence of imperative objects. FSTTCS 1997.

(Red Object) $(\mathcal{R}[o], \sigma) \rightarrow (\mathcal{R}[\iota], \sigma')$ if $\sigma' = (\iota \mapsto o) :: \sigma$ and $\iota \notin \text{dom}(\sigma)$.

(Red Select) $(\mathcal{R}[\iota.\ell_j], \sigma) \rightarrow (\mathcal{R}[b_j \{\!\!\{ \iota/x_j \}\!\!\}], \sigma)$
if $\sigma(\iota) = [\ell_i = \varsigma(x_i)b_i]_{i \in 1..n}$ and $j \in 1..n$.

(Red Update) $(\mathcal{R}[\iota.\ell_j \Leftarrow \varsigma(x)b], \sigma) \rightarrow (\mathcal{R}[\iota], \sigma')$
if $\sigma(\iota) = [\ell_i = \varsigma(x_i)b_i]_{i \in 1..n}$, $j \in 1..n$, and
 $\sigma' = \sigma + (\iota \mapsto [\ell_i = \varsigma(x_i)b_i]_{i \in 1..j-1}, \ell_j = \varsigma(x)b, \ell_i = \varsigma(x_i)b_i]_{i \in j+1..n})$.

(Red Clone) $(\mathcal{R}[\text{clone}(\iota)], \sigma) \rightarrow (\mathcal{R}[\iota'], \sigma')$
if $\sigma(\iota) = o$, $\sigma' = (\iota' \mapsto o) :: \sigma$ and $\iota' \notin \text{dom}(\sigma)$.

(Red Let) $(\mathcal{R}[\text{let } x = v \text{ in } b], \sigma) \rightarrow (\mathcal{R}[b \{\!\!\{ v/x \}\!\!\}], \sigma)$.

(Red Appl) $(\mathcal{R}[(\lambda(x)b)(v)], \sigma) \rightarrow (\mathcal{R}[b \{\!\!\{ v/x \}\!\!\}], \sigma)$.

Figure: Small-step operational semantics

What kind of semantics for functions is this?

Conclusion

We had a look at many semantics to specify function behaviour; difficulty came from the imperative aspect + scopes + context switch.

Problems solved:

- stacking environment
- scope of variables
- restoring the right environment after invocation
- return a value

Each of the semantics has its advantages and corresponds to some of the semantics encountered in the literature.

We have not seen how to deal with imbricated scopes in general (it is often a simple extension along the same principles)